

Chat via UDP: Confirmação de Entrega e Reenvio Manual em Nível de Aplicação

Relatório da Atividade Prática — Redes de Computadores

1 Introdução

Esta atividade teve como objetivo implementar um serviço simplificado de mensagens instantâneas sobre o protocolo UDP (`SOCK_DGRAM`), no qual a própria aplicação — e não o sistema operacional ou a pilha de rede — é responsável por identificar mensagens, confirmar entregas e retransmitir pacotes perdidos. O canal de comunicação simula perda artificial de pacotes (`DROP_RATE = 0.4`), exigindo que o cliente mantenha um controle próprio de pendências.

2 Estratégia Adotada

A estratégia de controle foi dividida entre o remetente (`chat_sender.py`) e o destinatário (`chat_receiver.py`):

- **Identificação das mensagens:** cada mensagem enviada recebe um identificador sequencial incremental (`msg_counter`), garantindo que remetente e destinatário consigam referenciar univocamente cada pacote trocado.
- **Controle de pendências:** o remetente mantém um dicionário `pending_messages = {id: texto}`. Toda mensagem enviada é imediatamente registrada nesse dicionário com status *Pendente (Check Cinza)*, antes mesmo de se saber se ela chegou ao destino.
- **Confirmação assíncrona:** uma *thread* dedicada (`listen_receipts`) fica bloqueada em `recvfrom` aguardando recibos `DELIVERED|<ID>`. Ao receber um recibo válido, a *thread* remove o ID correspondente de `pending_messages` e exibe a confirmação *Entregue (Check Azul)*. Um `Lock` protege o dicionário compartilhado contra condições de corrida entre a *thread* de escuta e a *thread* principal (que insere novas mensagens).
- **Descarte simulado:** no destinatário, cada datagrama recebido é submetido a um sorteio (`random.random() < DROP_RATE`). Se o pacote for descartado, ele é simplesmente ignorado e nenhum recibo é enviado — reproduzindo fielmente uma perda de rede real, invisível ao remetente até que o *timeout* de confiança da aplicação (neste caso, a ausência de confirmação) seja percebido pelo usuário.

- **Reenvio manual:** o comando `/reenviar` percorre todas as entradas ainda presentes em `pending_messages` e as retransmite com o mesmo ID original, permitindo que o usuário force a recuperação de mensagens perdidas sem duplicar identificadores.
- **Visibilidade do estado:** o comando `/status` exibe a lista de mensagens pendentes, dando transparência ao usuário sobre quais mensagens ainda não foram confirmadas pelo destinatário.

3 Evidências de Execução

3.1 Captura de Tela 1: Envio com Descarte e Pendências

A Figura 1 mostra o cliente enviando as mensagens de IDs 1, 2 e 3. A mensagem de ID 1 foi confirmada normalmente, mas as mensagens de IDs 2 e 3 não retornaram recibo. Ao executar o comando `/status`, o cliente lista corretamente as duas mensagens ainda pendentes:

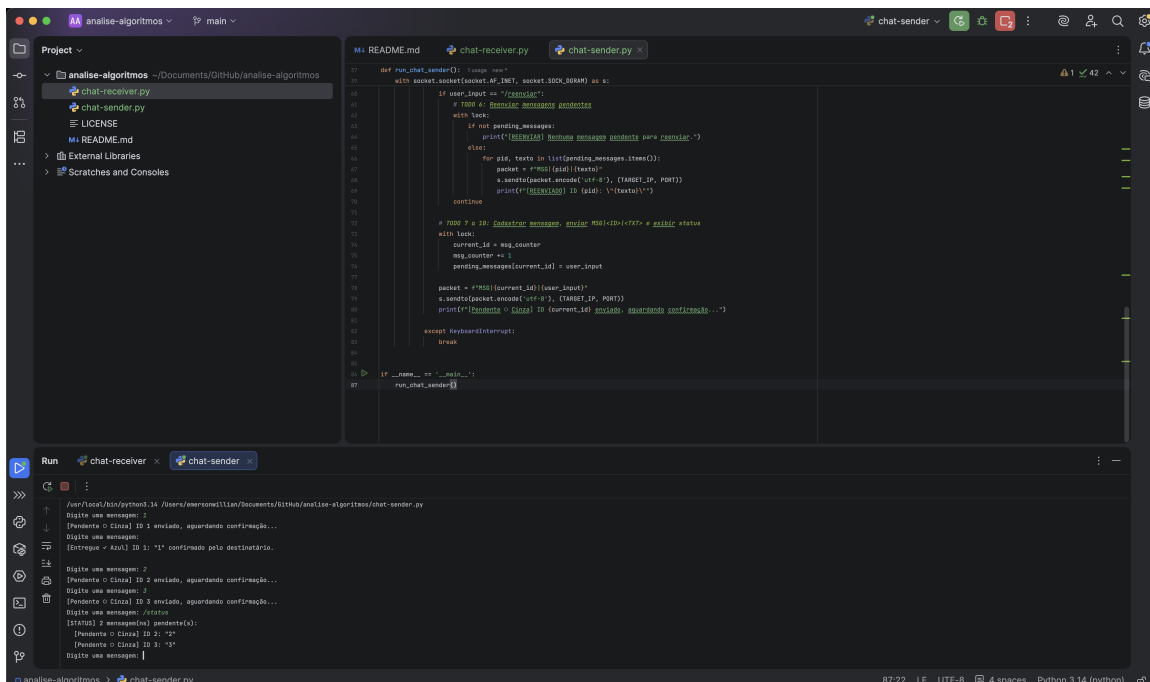


Figura 1: Cliente (`chat_sender.py`) enviando mensagens e exibindo pendências via `/status`.

A Figura 2 confirma, do lado do servidor, que os pacotes correspondentes aos IDs 2 e 3 foram efetivamente descartados pelo canal simulado, o que justifica a ausência dos recibos observada no cliente:

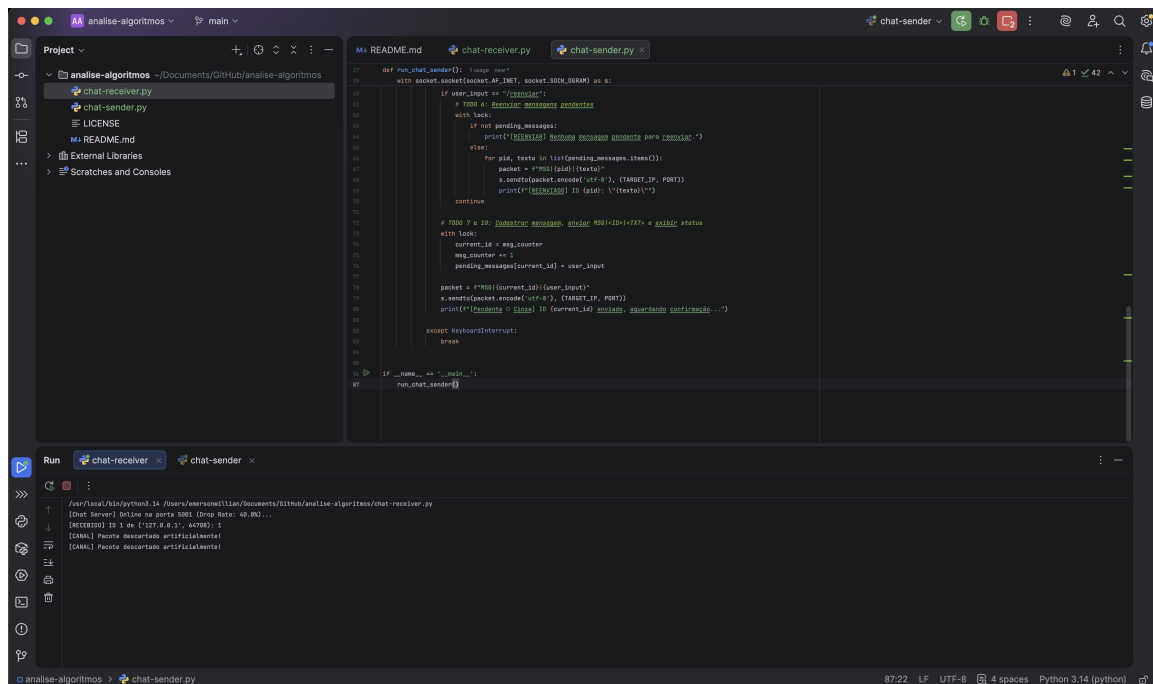


Figura 2: Servidor (chat_receiver.py) descartando artificialmente os pacotes referentes aos IDs 2 e 3.

3.2 Captura de Tela 2: Reenvio e Confirmação das Pendências

A Figura 3 mostra a execução do comando /reenviar. As mensagens de IDs 2 e 3, que continuavam em `pending_messages`, foram retransmitidas e, desta vez, ambas alcançaram o destinatário com sucesso, sendo confirmadas com o status *Entregue (Check Azul)*:

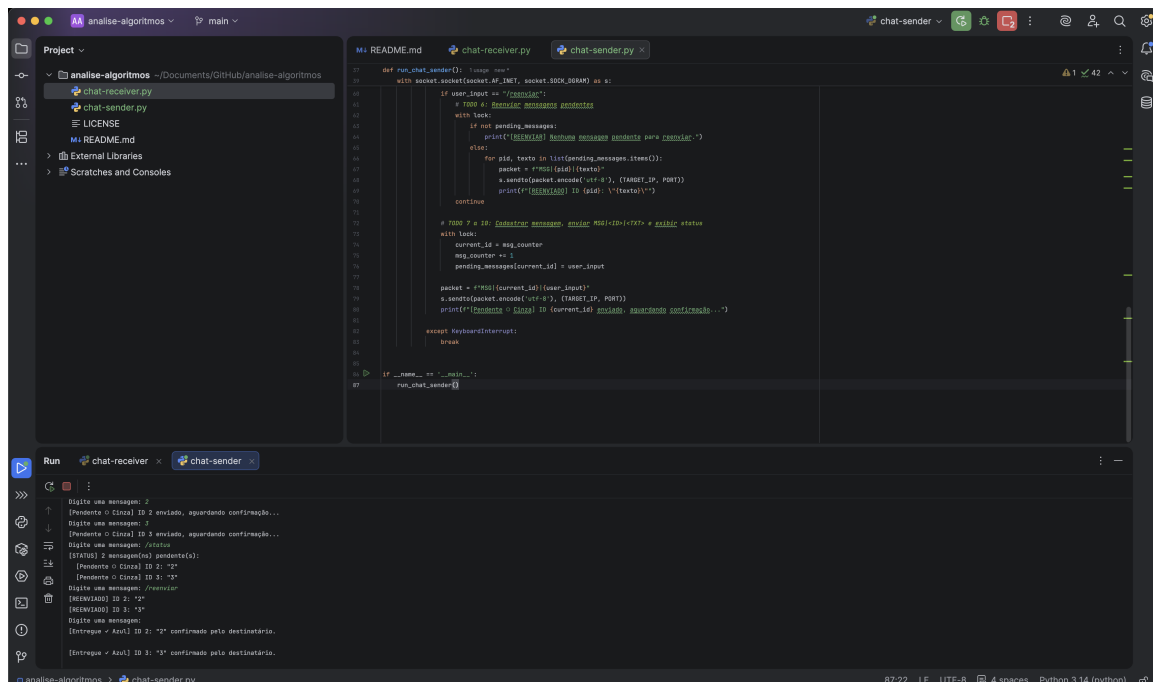


Figura 3: Execução do comando /reenviar destravando e confirmando a entrega das mensagens pendentes (IDs 2 e 3).

4 Discussão: Controle na Camada de Aplicação vs. Confiabilidade Nativa do TCP

O TCP resolve o problema de confiabilidade de forma transparente ao desenvolvedor: números de sequência, ACKs, janelas deslizantes, retransmissão automática por *timeout* e controle de congestionamento são geridos inteiramente pela pilha de protocolos do sistema operacional. A aplicação apenas escreve e lê bytes em um fluxo confiável, sem precisar se preocupar com perdas, duplicatas ou reordenação de pacotes.

Ao implementar esse controle manualmente sobre UDP, como feito nesta atividade, a aplicação ganha flexibilidade e visibilidade: é possível decidir *quando* e *quais* mensagens retransmitir (aqui, por comando explícito do usuário, e não por um temporizador automático), o que é útil em cenários como mensageiros instantâneos, jogos em tempo real ou streaming, nos quais uma retransmissão automática e cega poderia introduzir atraso desnecessário ou reenviar dados já obsoletos.

Em contrapartida, esse controle exige que o desenvolvedor reimplemente, mesmo que de forma simplificada, mecanismos que o TCP já oferece prontos: identificação de mensagens, confirmação de recebimento, temporização e lógica de retransmissão. Isso aumenta a complexidade do código da aplicação e traz o risco de bugs sutis de concorrência (por isso o uso de Lock sobre `pending_messages`), além de não lidar automaticamente com duplicatas, reordenação ou controle de congestionamento — problemas que o TCP resolve nativamente, mas que ficariam a cargo da aplicação caso o cenário exigisse.

Em suma, a atividade evidencia o *trade-off* clássico entre UDP e TCP: menor overhead de conexão e maior controle granular sobre a confiabilidade (UDP + lógica de aplicação) versus simplicidade de desenvolvimento e garantias automáticas de entrega ordenada (TCP).